MLR 570 · PROJECT GUIDE

Priority-Constrained Descent for Continual Learning

What the method is, what we are testing, and what has run so far

Faris Nasser · Salah Moussa

Instructor: Dr. Mohamed AlHajri · American University of Sharjah · Fall 2026

Status as of 5 November 2026

How to read this. red = key definitions and results blue = why it matters teal = mental model
amber = careful: limits, caveats, what we must not claim gray = worked example
Every experimental number names the experiment it came from (F-0xx). The full tables are in `docs/reports/explore_2026-11-05.md` and every run is a row of `results/ledger.csv`.

Every result in this guide is exploration-stage

All numbers come from protocol v0: the 10% validation split, one untuned learning rate (0.1) for every method, and at most 3 seeds. They decide which configurations get promoted. None of them is a reportable result yet.

Chapters: 1 The project in one page · 2 Background you need · 3 How PCD works · 4 Our study: PCD in continual learning · 5 What we built · 6 What ran and what it shows · 7 What is next · Quick reference

Contents

1 The Project in One Page 4

2 Background You Need 6

2.1 The dot product tells you whether two directions agree 6

2.2 Gradient descent, seen through the first-order picture 6

2.3 Continual learning: the setting 7

2.4 How continual learning is measured: the accuracy matrix 7

2.5 Why forgetting happens, in one line 8

2.6 The existing fixes 9

3 How PCD Works 10

3.1 The idea: anchor to the primary, distort the minimum 10

3.2 The PCD problem 10

3.3 The picture, and the closed form for one constraint 11

3.4 Normalization: why τ means the same thing for every loss 12

3.5 The rescale step 13

3.6 The threshold: when protecting the old task stalls the new one 13

3.7 Many old tasks: the general solution 14

3.8 The algorithm in plain steps 14

3.9 What PCD promises, and what it does not 15

4 Our Study: PCD in Continual Learning 17

4.1 The research question 17

4.2 How PCD maps onto continual learning 17

4.3 The four research objectives 18

4.4 How we test: the experimental ladder 19

5 What We Built, and How It Was Checked 20

6 What Ran, and What It Shows 21

6.1 What ran 21

6.2 Split MNIST: the overall picture 21

6.3 Finding 1: the margin and replay in the primary do the same job 22

6.4 Finding 2: PCD vs GEM's own knob 23

6.5 Finding 3: on CIFAR-100, PCD is about equal to ER 23

6.6 Scouting for Salah's objectives (1 seed, not for reporting) 24

6.7 Problems found and handled 24

7 What Is Next 25

7.1 Open questions for Dr. AlHajri 25

7.2 Next runs 25

7.3 Timeline 25

Quick reference 26

CHAPTER 1

1 · The Project in One Page

Read this first. No formulas.

The flow: catastrophic forgetting → the professor's method (PCD) → PCD applied to continual learning → four research objectives → what has run → what is next.

The problem. A neural network trained on task A and then on task B usually forgets A. This is *catastrophic forgetting*. Continual learning (CL) is the field that tries to stop it: one model learns a sequence of tasks and should stay good at all of them.

The professor's method. Dr. AlHajri and Dara Varam (our TA) published *Priority-Constrained Descent* (PCD) in TMLR 2026 [1]. PCD handles training with several losses where one loss matters most (the *primary*) and the others are conditions it must respect (the *secondaries*). At every step PCD moves as close as possible to the primary's direction, but only along directions that still give every secondary a guaranteed share of first-order progress (for small enough steps). One dial, $\tau \in [0, 1]$, sets how big that share is. The paper tested PCD on model compression, not on continual learning.

Our project. We apply PCD to continual learning. The new task is the primary, and each old task is a secondary that must not be pushed backwards. This is project idea #3 from the course list.

What is not new, and what is. Constraining the new task's step so that it does not hurt old tasks already exists: GEM (2017), A-GEM (2019) and ε-SOFT-GEM (2020). PCD with $\tau = 0$ *is* GEM. So we do not claim a new method. Our contribution is a **controlled study**: which parts of PCD actually change continual-learning behaviour, in which setting, and at what cost. "It does not help" is a valid answer.

Who does what.

RO	Question	Owner	Status
RO1	Is our code really PCD?	Shared	**verified**
RO2	Does PCD's margin τ help when we keep a replay memory?	Faris	exploring
RO3	Can PCD replace EWC's penalty weight when there is no memory?	Salah	scouting
RO4	What should PCD's running statistics do when the task changes?	Salah	scouting

Where we are (5 Nov).

- The method is implemented and checked against the official PCD code (9 of 9 cases identical) and against hand-derived answers. 277 automated checks pass.
- 227 runs have finished on Faris's machine: 226 exploration runs on Split MNIST and Split CIFAR-100, plus one plumbing smoke test (`F-000`).

- The main early finding: on Split MNIST, PCD's margin and putting replay into the main objective do the same job. PCD's best configuration beats GEM's best by a small but consistent margin. On CIFAR-100, PCD is about equal to plain replay.
- Salah's axes (RO3, RO4) have only 1-seed scouting runs, done by Faris. His own runs start next.
- The paper is not started. It needs reportable results first.

CHAPTER 2

2 · Background You Need

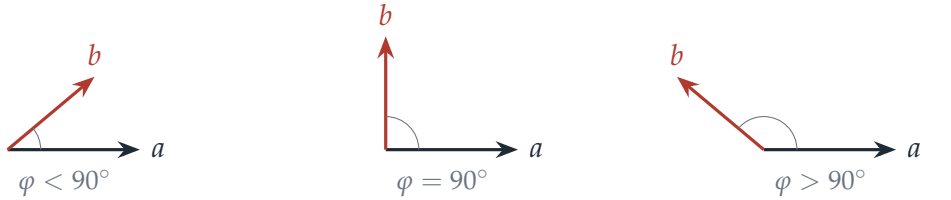
Dot products, gradient descent, and how continual learning is measured.

2.1 The dot product tells you whether two directions agree

For two vectors $a, b \in \mathbb{R}^n$,

$$a^\top b \;=\; \sum_{k=1}^{n} a_k b_k \;=\; \|a\|\,\|b\| \cos\varphi, \tag{1}$$

where $\|a\| = \sqrt{a^\top a}$ is the length of a and φ is the angle between a and b. Lengths are never negative, so the **sign** of $a^\top b$ is the sign of $\cos\varphi$:



Sign of the dot product $a^\top b > 0$ means a and b point into the same half of space. $a^\top b < 0$ means they point more than 90° apart.

Two facts we use later. If $\|a\| = \|b\| = 1$, then $a^\top b = \cos\varphi$ exactly. And the *projection* of b onto the direction of a is $\frac{a^\top b}{\|a\|^2}\, a$: the part of b that lies along a.

2.2 Gradient descent, seen through the first-order picture

Training minimizes a loss $L(\theta)$ over the parameters $\theta \in \mathbb{R}^n$. Let $g = \nabla L(\theta)$ be the gradient. If we move the parameters by a small step Δ, the first-order Taylor expansion says

$$L(\theta + \Delta) \;\approx\; L(\theta) + g^\top \Delta. \tag{2}$$

Gradient descent takes $\Delta = -\eta\, d$ with a step size $\eta > 0$ and a direction d (plain SGD uses $d = g$). Then the change in loss is

$$L(\theta - \eta d) - L(\theta) \;\approx\; -\eta\, g^\top d. \tag{3}$$

First-order descent A direction d lowers L for a small enough step if $g^\top d > 0$, and raises it if $g^\top d < 0$ (at $g^\top d = 0$ the first-order term says nothing). The bigger $g^\top d$, the more progress per unit of step. With $d = g$ we get $g^\top g = \|g\|^2 \geq 0$, which is why following the gradient works.

Careful

"Small enough" is doing real work. Equation (2) ignores curvature. If the loss is β-smooth (its gradient changes by at most β per unit of distance), a step $\eta < 2/\beta$ along $d = g \neq 0$ is guaranteed to lower it; for a general d with $g^\top d > 0$ the condition is $\eta < 2g^\top d/(\beta\|d\|^2)$. With a large step, a positive dot product promises nothing. Every "guarantee" in this guide is first-order in this sense.

2.3 Continual learning: the setting

A model sees tasks $1, 2, \ldots, T$ one after another. While training on task t it cannot see the full data of earlier tasks. Two ways to keep something from the past:

- a small **replay memory** $\mathcal{M}$ of stored examples from old tasks (*rehearsal* methods);
- a **penalty** that discourages moving parameters that mattered for old tasks (*regularization* methods, e.g. EWC).

Class-incremental vs task-incremental. In Split MNIST the 10 digits are cut into 5 tasks of 2 digits each. At test time:

- **class-incremental (class-IL)**: the model must pick the right digit out of all 10. This is hard, and it is our headline setting.
- **task-incremental (task-IL)**: the model is told which task the image comes from, so it only chooses between that task's 2 digits. This is much easier.

Our framework (Mammoth) reports both from the same run.

Why it matters A model that forgets everything except the last task gets about 2/10 of the digits right in class-IL. That is why fine-tuning sits near 19.7% on Split MNIST (F-001): the "floor" of class-IL.

2.4 How continual learning is measured: the accuracy matrix

After finishing training on task t, we test on every task i. That gives the **accuracy matrix** $R_{t,i}$: row t = "after training task t", column i = "tested on task i". All metrics come from R, and only one script computes them (`eval/evaluate.py`).

The four metrics

$$A_T = \frac{1}{T}\sum_{i=1}^{T} R_{T,i} \qquad \text{final average accuracy (higher is better)} \tag{4}$$

$$F_T = \frac{1}{T-1}\sum_{i=1}^{T-1}\Big(\max_{t\in\{i,\dots,T-1\}} R_{t,i} - R_{T,i}\Big) \qquad \text{average forgetting (lower is better)} \tag{5}$$

$$\mathrm{LA} = \frac{1}{T}\sum_{i=1}^{T} R_{i,i} \qquad \text{learning accuracy: plasticity} \tag{6}$$

$$\mathrm{BWT} = \frac{1}{T-1}\sum_{i=1}^{T-1}\big(R_{T,i} - R_{i,i}\big) \qquad \text{backward transfer (GEM's definition)} \tag{7}$$

In words: A_T is how good the final model is on everything. F_T is how far each old task fell from its best. LA is how well each task was learned when it was new. A method can lower forgetting by learning each new task less well, which is why we always look at F_T and LA together.

Worked example

Three tasks. After each task we test on the tasks seen so far:

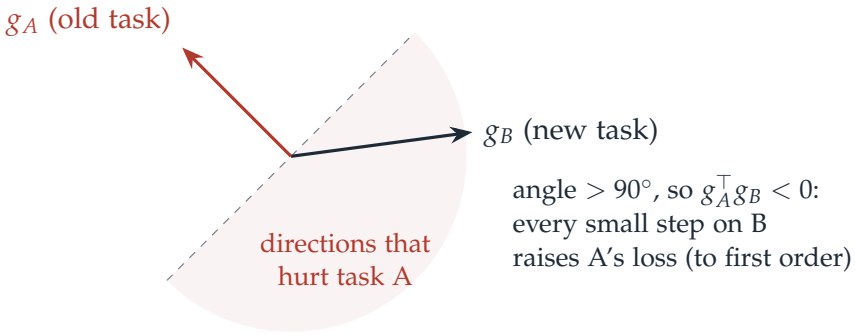
$$R = \begin{bmatrix} 95 & \cdot & \cdot \\ 70 & 96 & \cdot \\ 60 & 80 & 97 \end{bmatrix}.$$

$A_3 = (60+80+97)/3 = 79.0$. $\mathrm{LA} = (95+96+97)/3 = 96.0$. Forgetting: task 1 fell from its best 95 to 60 (35 points), task 2 from 96 to 80 (16 points), so $F_3 = (35+16)/2 = 25.5$. Here $\mathrm{BWT} = -25.5$ too, because each task's best was the moment it was learned.

2.5 Why forgetting happens, in one line

Let g_A be the old task's gradient and g_B the new task's. Training on B steps along $d = g_B$. By (3), the old task's loss changes by

$$\Delta L_A \;\approx\; -\eta\, g_A^\top g_B. \tag{8}$$



Mental model If the two gradients agree ($g_A^\top g_B > 0$), learning B also helps A. If they conflict ($g_A^\top g_B < 0$), every small step on B undoes some of A (to first order). Forgetting is a sign problem of a dot product. Every method in this guide is a way of controlling that

sign.

2.6 The existing fixes

Method	What it does	Formula (one step)
Fine-tuning	Nothing: just train on the new task. Lower reference.	$d = \nabla L_t$
Joint	Train on all tasks' data at once. Offline ceiling, not a CL method.	$d = \nabla \sum_i L_i$
ER [7]	Mix a replay batch from memory into every step.	$d = \nabla \left[L_t + L_{\mathcal{M}} \right]$
EWC [6]	Penalize moving weights that mattered for old tasks. F_k is the Fisher information (how much weight k mattered), θ^* the weights at the end of the old task.	$L_t + \frac{\lambda}{2} \sum_k F_k (\theta_k - \theta_k^*)^2$
GEM [3]	Project the new step so that no old task's memory loss rises: one constraint per old task.	$\min_d \lVert d - g \rVert^2$ s.t. $g_j^\top d \geq 0$
A-GEM [4]	Cheaper GEM: one constraint on the average memory gradient.	same, with one g_{ref}
ε-SOFT-GEM [5]	A-GEM with a positive, normalized margin ε.	one constraint, margin ε

Why it matters EWC mixes the two goals with a weight λ whose right value depends on how big the penalty is compared with the task loss. That changes with the model, the data and the number of tasks, so λ must be re-tuned. GEM's constraint $g_j^\top d \geq 0$ only forbids hurting old tasks. It does not ask them to improve. PCD addresses both.

CHAPTER 3

3 · How PCD Works

The professor's method, step by step. Inspired by the PCD project page (daravaram.github.io/PCD) and the paper.

The flow: the idea (anchor to the primary) → the QP → its picture and closed form → normalization → rescale → the threshold → many secondaries → the algorithm → what it does and does not promise.

3.1 The idea: anchor to the primary, distort the minimum

Most multi-objective methods treat every loss as an equal peer: they average the gradients, or search for weights. PCD instead says one loss is the deliverable (the *primary*, gradient g_0) and the others are conditions (the *secondaries*, gradients $g_1, \dots, g_k$).[1] The rule:

Mental model Go where the primary wants to go. Bend only as much as needed so that every secondary still gets a guaranteed share of progress. If the primary already gives every secondary its share, do not bend at all.

3.2 The PCD problem

Each step solves a small quadratic program (QP) on the *normalized* gradients $\tilde{g}_i$ (Section 3.4 explains the tilde):

$$\tilde{d}^* \;=\; \arg\min_{\tilde{d}\in\mathbb{R}^n} \tfrac{1}{2}\|\tilde{d}-\tilde{g}_0\|^2 \quad \text{s.t.} \quad \tilde{g}_j^\top \tilde{d} \;\geq\; \tau\,\|\tilde{g}_j\|^2, \qquad j=1,\dots,k. \tag{9}$$

Read it piece by piece:

- $\frac{1}{2}\|\tilde{d}-\tilde{g}_0\|^2$: stay as close as possible to the primary's direction.
- $\tilde{g}_j^\top \tilde{d}$: the first-order progress the step gives secondary j, by (3).
- $\tau\|\tilde{g}_j\|^2$: the progress secondary j would get if we followed its own gradient ($\tilde{g}_j^\top \tilde{g}_j = \|\tilde{g}_j\|^2$), times τ. So each constraint says: *give secondary j at least a fraction τ of the progress it would get on its own.*
- $\tau = 0$: the constraint becomes $\tilde{g}_j^\top \tilde{d} \geq 0$, "do not hurt j". With raw gradients that is **exactly GEM**.

One dial $\tau \in [0,1]$ is the only knob. $\tau = 0$ only forbids first-order harm to the secondaries. Larger τ demands that they actively improve. In continual learning, τ is the stability–plasticity dial.

[1] The paper numbers the primary 1 and the secondaries $2, \dots, K$. We number the primary 0, so that secondary j is past task j.

3.3 The picture, and the closed form for one constraint

Each constraint is a half-space: the set of directions on one side of a "wall" perpendicular to $\tilde{g}_j$. Solving (9) means projecting $\tilde{g}_0$ onto the region inside all the walls. With one secondary the answer has a closed form.

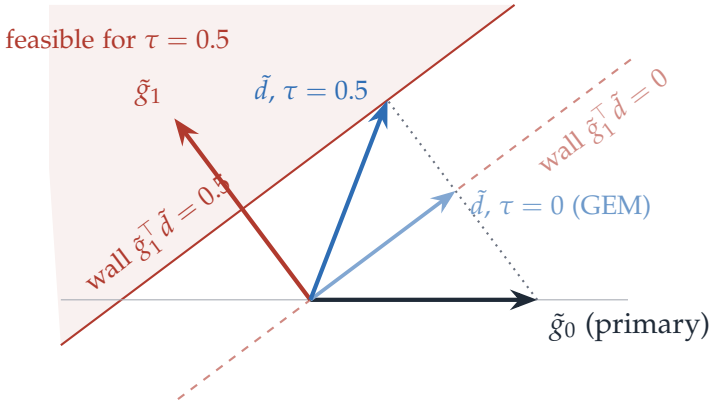


Figure 1: One old task, unit vectors, $\cos\varphi = -0.6$. The primary $\tilde{g}_0$ points into the forbidden side. PCD moves it along $\tilde{g}_1$ (dotted) until it reaches the wall. $\tau = 0$ puts the wall through the origin (GEM). $\tau = 0.5$ moves the wall out, so $\tilde{d}$ bends further toward the old task. The numbers are worked out in the example below.

Deriving it. Write the answer as $\tilde{d} = \tilde{g}_0 + \lambda\tilde{g}_1$ with $\lambda \geq 0$: we only ever add some of $\tilde{g}_1$, never subtract (this is what the KKT conditions of (9) give; Section 3.7). If $\tilde{g}_0$ already satisfies the wall, take $\lambda = 0$. Otherwise, choose λ to land exactly on the wall:

$$\tilde{g}_1^\top(\tilde{g}_0 + \lambda\tilde{g}_1) = \tau\|\tilde{g}_1\|^2 \quad \Longrightarrow \quad \lambda = \frac{\tau\|\tilde{g}_1\|^2 - \tilde{g}_1^\top\tilde{g}_0}{\|\tilde{g}_1\|^2}.$$

Both cases together:

One-constraint closed form (PCD Cor. 4.6)

$$\tilde{d} = \tilde{g}_0 + \lambda\,\tilde{g}_1, \qquad \lambda = \max\left(0,\ \frac{\tau\|\tilde{g}_1\|^2 - \tilde{g}_1^\top\tilde{g}_0}{\|\tilde{g}_1\|^2}\right). \tag{10}$$

For unit vectors, with $c = \cos\varphi = \tilde{g}_0^\top\tilde{g}_1$, this is simply

$$\lambda = \max(0,\ \tau - c). \tag{11}$$

Worked example: one wall, by hand

Take $\tilde{g}_0 = (1, 0)$ and $\tilde{g}_1 = (-0.6,\ 0.8)$. Both have length 1, and $c = \tilde{g}_0^\top\tilde{g}_1 = -0.6$: the tasks conflict.

- $\tau = 0$ **(GEM).** $\lambda = 0 - (-0.6) = 0.6$, so $\tilde{d} = (1, 0) + 0.6(-0.6, 0.8) = (0.64,\ 0.48)$.
  Check: $\tilde{g}_1^\top\tilde{d} = -0.384 + 0.384 = 0$, exactly on the wall. Primary progress $\tilde{g}_0^\top\tilde{d} = 0.64$.
- $\tau = 0.5$. $\lambda = 0.5 + 0.6 = 1.1$, so $\tilde{d} = (1 - 0.66,\ 0.88) = (0.34,\ 0.88)$.
  Check: $\tilde{g}_1^\top\tilde{d} = -0.204 + 0.704 = 0.5$. Primary progress $\tilde{g}_0^\top\tilde{d} = 0.34$.

- $\tau = 1$. $\lambda = 1.6$, $\tilde{d} = (0.04, 1.28)$. Primary progress 0.04, almost nothing left for the new task.
- **No conflict.** If instead $\tilde{g}_1 = (0.6, 0.8)$, then $c = 0.6 \geq \tau = 0.5$: the wall is already satisfied, $\lambda = 0$ and $\tilde{d} = \tilde{g}_0$. PCD does not bend.

These are the numbers in our toy figure (primary progress 0.64, 0.34, 0.04 at $\tau = 0, 0.5, 1$; `toy/results.json`).

3.4 Normalization: why τ means the same thing for every loss

If one loss is 100 times bigger than another, its gradient is 100 times longer, and "a fraction τ of progress" would mean very different things for the two. PCD therefore divides each gradient by a running estimate of its own typical length. For each objective i, at its s-th update (each objective keeps its own counter, so an old task that joins the constraints later starts at $s = 1$):

$$v_{i,s} = \beta\, v_{i,s-1} + (1-\beta)\, \|g_{i,s}\|^2, \qquad v_{i,0} = 0, \tag{12}$$

$$\hat{v}_{i,s} = \frac{v_{i,s}}{1-\beta^s}, \tag{13}$$

$$\tilde{g}_i = \frac{g_{i,s}}{\sqrt{\hat{v}_{i,s} + \varepsilon}}, \tag{14}$$

with $\beta = 0.999$ and $\varepsilon = 10^{-8}$ (the paper's defaults).

- (12) is an exponential moving average (EMA) of the squared gradient length. $\beta = 0.999$ means it remembers roughly the last $1/(1-\beta) = 1000$ steps.
- (13) is the bias correction. Because v starts at 0, early values are too small. If every $\|g\|^2$ were the same number q, then $v_s = (1-\beta^s)\, q$; dividing by $1-\beta^s$ gives back exactly q.
- (14) divides by the typical length, so in steady state $\|\tilde{g}_i\| \approx 1$ for every objective.

Asymptotic scale invariance (PCD Thm. B.9) Multiply a loss by any $c > 0$. Its gradient becomes $c\,g$, its EMA becomes $c^2\hat{v}$, and

$$\frac{c\,g}{\sqrt{c^2\hat{v} + \varepsilon}} \approx \frac{g}{\sqrt{\hat{v} + \varepsilon}} \qquad (\text{when } c^2\hat{v} \gg \varepsilon).$$

The normalized gradient does not change, so the PCD direction does not change. τ keeps its meaning whatever the raw scale of each loss.

Careful

This holds only while $c^2\hat{v} \gg \varepsilon$. When a gradient becomes tiny (of order $\sqrt{\varepsilon} = 10^{-4}$), ε dominates the denominator and the invariance breaks (PCD Rem. B.11). This matters for EWC-style penalties, whose gradient is zero at the old task's weights. Our scouting run saw exactly this (F-011, Section 6.6).

3.5 The rescale step

The QP gives a direction $\tilde{d}$ in normalized units. PCD then gives it the primary's raw length and takes a plain SGD step:

$$d = \tilde{d}\,\frac{\|g_0\|}{\|\tilde{d}\|}, \qquad \theta \leftarrow \theta - \eta\, d. \tag{15}$$

Multiplying by a positive number never changes the sign of a dot product, so d keeps the sign of every $\tilde{g}_j^\top \tilde{d}$. The margin itself is scaled by $\|g_0\| / \|\tilde{d}\|$, so for $\tau > 0$ the wall $\tilde{g}_j^\top d \geq \tau \|\tilde{g}_j\|^2$ need not hold for d: the τ-guarantee is about $\tilde{d}$ (PCD §4.7). If $\tilde{d} = 0$, PCD sets $d = 0$ (Alg. 1, line 18).

Careful

Plain SGD only. Adam rescales each coordinate differently, which can flip the signs PCD worked to guarantee (PCD §4.7). That is why every method in our study uses plain SGD.

3.6 The threshold: when protecting the old task stalls the new one

When the wall is active, how much progress is left for the primary? With unit vectors and (11),

$$\tilde{g}_0^\top \tilde{d} = \tilde{g}_0^\top (\tilde{g}_0 + \lambda \tilde{g}_1) = 1 + (\tau - c)\, c = 1 - c^2 + \tau c. \tag{16}$$

For conflicting tasks ($c < 0$) this falls as τ grows, and it turns negative when

$$\boxed{\tau > \frac{1 - c^2}{-c}} \qquad \text{(PCD Prop. 4.7).} \tag{17}$$

Past this threshold, protecting the old task makes the new task's loss go up (to first order). The threshold is below 1, so reachable with $\tau \in [0, 1]$, only when $1 - c^2 < -c$, that is when $c < (1 - \sqrt{5})/2 \approx -0.62$: only for strongly conflicting tasks.

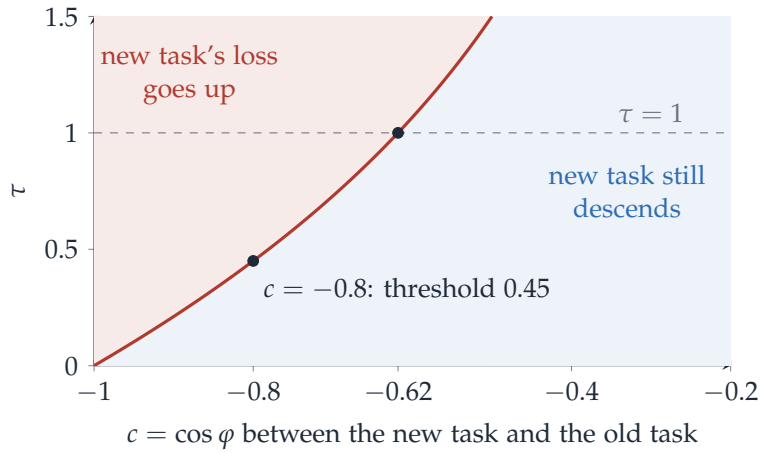


Figure 2: Prop. 4.7 for unit vectors. Below the red curve the step still lowers the new task's loss. Above it, the old task is protected at the new task's expense.

Worked example: our toy problem

In our 2-D toy (`toy/run_toy.py`) the first step has $c = -0.8006$, so the threshold is $(1 - 0.8006^2)/0.8006 = 0.4485$. The solver's primary progress at each τ:

τ	0	0.1	0.25	0.5	1
$\tilde{g}_0^\top \tilde{d}$	0.359	0.279	0.159	−0.041	−0.442

It turns negative between 0.25 and 0.5, as the threshold predicts. The QP's answer equals the hand formula (10) with maximum difference 0.0.

Why it matters In continual learning τ is a stability–plasticity trade-off. A τ that is safe when tasks conflict mildly can stall learning when they conflict strongly. We log the fraction of steps where this happens (the *primary-ascent rate*).

3.7 Many old tasks: the general solution

With k old tasks there are k walls. The KKT conditions of (9) (the optimality conditions for constrained problems) say the solution always has the form

$$\tilde{d} = \tilde{g}_0 + \sum_{j=1}^{k} \lambda_j \tilde{g}_j, \qquad \lambda_j \geq 0, \tag{18}$$

with *complementary slackness*: $\lambda_j > 0$ only if wall j is active (met with equality); a wall that is already satisfied gets $\lambda_j = 0$. The primary's coefficient is pinned to 1. The secondaries only ever add corrections.

To find the λ_j, PCD solves the smaller *dual* problem on the $k \times k$ Gram matrix $M_{ij} = \tilde{g}_i^\top \tilde{g}_j$, $i, j = 1, \ldots, k$ (not the memory size M of Chapter 4):

$$\min_{\lambda \geq 0} \ \tfrac{1}{2} \lambda^\top M \lambda - r^\top \lambda, \qquad r_j = \tau M_{jj} - \tilde{g}_j^\top \tilde{g}_0. \tag{19}$$

This is a problem in k unknowns (the number of old tasks), not in n (millions of parameters), so it is cheap.

When there is no solution. With $\tau > 0$ the walls can contradict each other: panel (c), where one old task wants $x \geq \tau$ and another $x \leq -\tau$. This happens only when the old tasks' gradients cancel, formally when 0 lies in their convex hull (PCD Thm. B.2). When there are fewer old tasks than parameters, this almost never happens (Cor. B.3). The paper's fallback is to drop the walls for that step. The alternative we study is to fall back to the $\tau = 0$ walls (GEM), which are always satisfiable by $\tilde{d} = 0$.

In our runs so far The infeasible rate was **0 in every run** (every PCD and PCD-EWC run: F-004, F-007, F-010–F-013, F-015, F-016), as Cor. B.3 predicts.

3.8 The algorithm in plain steps

One PCD step (PCD Alg. 1; our code: `src/pcd_cl/direction.py`, `qp.py`, `normalize.py`):

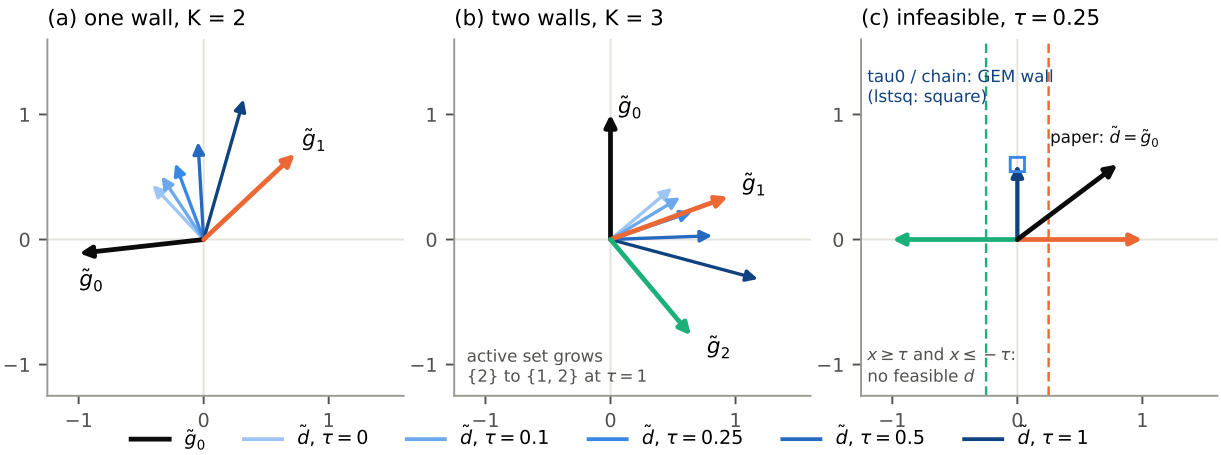


Figure 3: Our toy (`toy/run_toy.py`). (a) One wall: as τ grows, $\tilde{d}$ bends toward $\tilde{g}_1$. (b) Two walls: the set of active walls grows from $\{\tilde{g}_2\}$ to $\{\tilde{g}_1, \tilde{g}_2\}$ at $\tau = 1$. (c) Infeasible: two old tasks pointing in opposite directions, so no direction can give both positive progress.

1. Compute the raw gradients g_0 (primary) and $g_1, \ldots, g_k$ (one per old task).
2. Update each EMA (12), bias-correct (13), and normalize (14).
3. If $g_0 = 0$, set $d = 0$ (the primary-stationary halt). Otherwise, if $\tilde{g}_0$ already satisfies every wall, $\tilde{d} = \tilde{g}_0$.
4. Otherwise solve the dual (19) for λ and set $\tilde{d}$ by (18).
5. If the walls are infeasible, use the fallback.
6. If $\tilde{d} = 0$, set $d = 0$ (a fixed point, not convergence); otherwise rescale to the primary's length (15). Take a plain SGD step.

3.9 What PCD promises, and what it does not

What it gives

- When the QP is feasible, the normalized direction $\tilde{d}$ gives each secondary (its replay or penalty loss) at least a τ-fraction of first-order progress; the rescaled d keeps the sign of that progress.
- When $\tilde{g}_0$ already satisfies every wall, the step is exactly the primary's direction.
- τ has a scale-free meaning, thanks to normalization (while ε is negligible).
- One bounded dial instead of a search over loss weights.

What it does not give (we must never claim these)

- It does **not** guarantee "no forgetting". The walls are first-order and hold for small enough steps only.
- In CL the walls are computed on a few replay samples per old task. Satisfying them on those samples is not a guarantee on the old task's true loss.

- The guarantees are about the direction $\tilde{d}$ at one step, not a convergence theorem for the whole training run.
- Adam can break the sign guarantees, so we use plain SGD.

CHAPTER 4

4 · Our Study: PCD in Continual Learning

The research question, the two forms, and the four objectives.

4.1 The research question

Research question PCD's constraint is already known in continual learning: with $\tau = 0$ it is GEM. So the question is not *whether* constraint projection works in CL. It is: **which parts of PCD change continual-learning behaviour, in which setting, and at what cost?**

The parts of PCD that differ from the GEM line as published, and so the parts we study:

1. a normalized margin τ *per old task* (ε-SOFT-GEM has one margin on one averaged constraint; GEM's released code has a per-task γ, but as a floor on the dual multipliers, not a normalized progress margin);
2. a *buffer-free* form, where the secondaries come from EWC-style penalties instead of a replay memory;
3. what the EMA statistics should do when the task changes;
4. what to do if the walls ever become infeasible.

Novelty boundary

We never write that constraining against past-task gradients is new (GEM, A-GEM), or that a positive margin is new (ε-SOFT-GEM, and GEM's own released code). PCD is the professor's method; we apply and study it. A clean negative result is a valid outcome.

4.2 How PCD maps onto continual learning

While training on task t, the primary is the current task, and there is one secondary per old task $j < t$. There are two ways to build the secondaries:

	Rehearsal form (RO2, Faris)	**Regularization form** (RO3, Salah)
Memory	a replay buffer $\mathcal{M}_j$ per old task	none
Primary g_0	$\nabla\left[L_t + L_{\mathcal{M}}\right]$: current batch plus a replay batch (default), or ∇L_t only (ablation)	∇L_t
Secondary g_j	$\nabla L_j(\mathcal{M}_j)$, the loss on 32 stored samples of task j	$\nabla\Omega_j = c\,F_j \odot (\theta - \theta_j^*)$, the gradient of an EWC penalty
Closest baseline	GEM ($\tau = 0$, raw gradients, current-only primary); ER (no walls)	EWC ($L_t + \lambda \sum_j \Omega_j$)
Code	`src/mammoth_models/pcd.py`	`src/mammoth_models/pcd_ewc.py`

Here $\odot$ is element-wise multiplication, F_j is the diagonal Fisher information of task j, θ_j^* are the weights at the end of task j, and c is a penalty scale.

Why it matters Why the primary includes replay by default: the project description defines the main direction from the current task *together with* an aggregate of past knowledge, by analogy with FedPCD [2], whose primary is the aggregate over all participating clients (its client history enters only the secondaries; D-005). Whether that choice matters turned out to be our biggest early finding (Section 6.3).

4.3 The four research objectives

RO1 — Formalize and verify (shared). Status: verified. Show that PCD-CL reduces exactly to GEM, A-GEM and ε-SOFT-GEM in the right limits, and check our solver against hand-derived answers, the KKT conditions and the official PCD code.

RO2 — The rehearsal margin (Faris). Status: exploring.

H1 At a matched memory and update budget, PCD with a moderate τ (0.1–0.5) gives lower forgetting F_T than GEM and ER, while losing at most 2 points of learning accuracy.

Refuted if, on both Split MNIST and Split CIFAR-100, the validation-tuned $\tau > 0$ beats GEM by more than one paired standard deviation (3 seeds $\times$ 3 task orders) on neither F_T nor A_T.

RO3 — The regularization form (Salah). Status: scouting.

H2 (a) With an equal small tuning grid and an unknown penalty scale $c \in [10^{-2}, 10^{2}]$, PCD-EWC's τ transfers across scales while EWC's λ must be re-tuned, so PCD-EWC reaches a higher A_T. (b) Fully tuned, PCD-EWC matches or beats EWC.

Scale invariance itself is a theorem when $c^2\hat{v}_j \gg \varepsilon$ (Thm. B.9), so we verify it rather than test it, and log where the ε floor breaks it.

RO4 — Task-boundary dynamics (Salah leads). Status: scouting. At a task switch the primary's gradient size jumps. With $\beta = 0.999$ a carried-over EMA mis-normalizes it for about 10^3 steps. Three modes: `carry` (keep every EMA), `reset` (restart every EMA), `bank` (keep one EMA per old task, restart the primary's; this mirrors FedPCD's per-client history).

H3 `reset` or `bank` lowers the post-boundary normalization error (how far $\|\tilde{g}_0\|$ is from 1) and lowers F_T, compared with `carry`.

Refuted if no mode changes F_T by more than one standard deviation.

4.4 How we test: the experimental ladder

Rung	Setting	Purpose
1. Toy	2–3 quadratics in $\mathbb{R}^2$, every number checkable by hand	RO1: the solver is right
2. Split MNIST	5 tasks $\times$ 2 digits, MLP (2 hidden layers of 256), 1 epoch per task	fast sweeps for H1–H3
3. Split CIFAR-100	10 tasks $\times$ 10 classes, reduced ResNet-18	headline results, best configurations only

Fair-comparison rules (`docs/PROTOCOL.md`, still a v0 draft):

- plain SGD, batch 32, the same backbone, initialization and epochs for every method;
- matched memory ($M \in \{200, 500\}$ stored examples), counting everything stored;
- all tuning on a 10% validation split of each task's training data, never on the test set; baselines get the same tuning effort;
- 3 seeds for anything promoted; 3 class orders on CIFAR-100;
- the falsification criteria are written down *before* the reportable runs (`docs/PREREGISTRATION.md`).

Baselines. Fine-tuning, joint, ER, GEM, A-GEM, ε-SOFT-GEM, EWC, and DER++ [8] where compute allows. GEM's released code has a "memory strength" γ (default 0.5) that pushes the step further toward old tasks; $\gamma = 0$ is the paper's exact projection. We run both.

CHAPTER 5

5 · What We Built, and How It Was Checked

Every layer sits on the one before it.

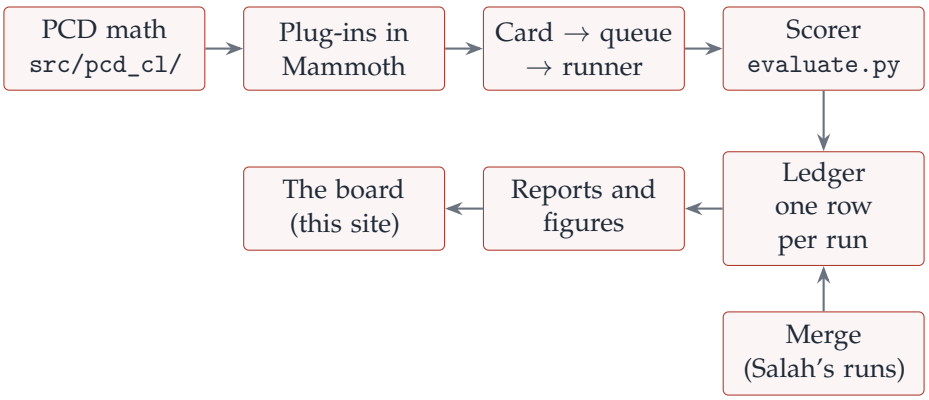


Figure 4: The pipeline. Nobody types numbers into the board: everything flows from the runs.

Layer	What and where	How we know it works
PCD math	QP solver, EMA normalizer, direction, fallbacks (`src/pcd_cl/`)	231 tests: closed forms, KKT, the GEM reduction, scale invariance. 9 of 9 cases identical to the official PCD code.
Toy	`toy/run_toy.py`	QP = hand formula (max difference 0); threshold crossed where Prop. 4.7 predicts.
Framework	Mammoth, a standard CL library, pinned to commit `e75a491`	Its baselines ran in every run; two upstream bugs found and handled.
Our methods	PCD, PCD-EWC, ε-SOFT-GEM (`src/mammoth_models/`)	28 plug-in tests; bit-exact reruns.
Plumbing	experiment cards, job queue, runner, logging	5 smoke tests; 226 runs plus 1 smoke test done; 1 run failed for a real reason.
Scorer	`eval/evaluate.py`, the only place metrics are computed	Its hash is checked on every merged run.
Results flow	Salah packs, Faris merges (`scripts/merge_results.py`)	4 tests: the merge rejects runs that are not comparable.

RO1 is verified 277 automated checks pass and 0 fail. Inside Mammoth, PCD with $\tau = 0$, a current-only primary and no rescale gives 60.3% on Split MNIST at $M = 200$ against 58.9% for GEM with $\gamma = 0$ (F-012 vs F-005): our plug-in reproduces GEM, as the math says it should.

CHAPTER 6

6 · What Ran, and What It Shows

226 exploration runs (plus one smoke test), 4–5 November 2026. Every number: validation split, lr 0.1, not reportable.

6.1 What ran

Exp	What it tested
F-001	references: fine-tuning and joint (Split MNIST)
F-002	ER, GEM ($\gamma = 0.5$), A-GEM at $M \in \{200, 500\}$
F-003	ε-SOFT-GEM, the closest prior art
F-004	**H1 core**: PCD τ sweep $\times$ normalization on/off $\times$ memory
F-005	GEM's exact projection ($\gamma = 0$)
F-006, F-007	CIFAR-100, 1 epoch per task: baselines and the PCD τ sweep
F-009–F-011	scouting for Salah, 1 seed: EWC λ grid, EMA boundary modes, PCD-EWC $\tau\times$ scale
F-012, F-013	ablation: which PCD ingredient matters (primary $\times$ rescale $\times$ τ), at $M = 200$ and 500
F-014	GEM γ grid
F-015, F-016	CIFAR-100 at 5 epochs per task, replay vs current-only primary

6.2 Split MNIST: the overall picture

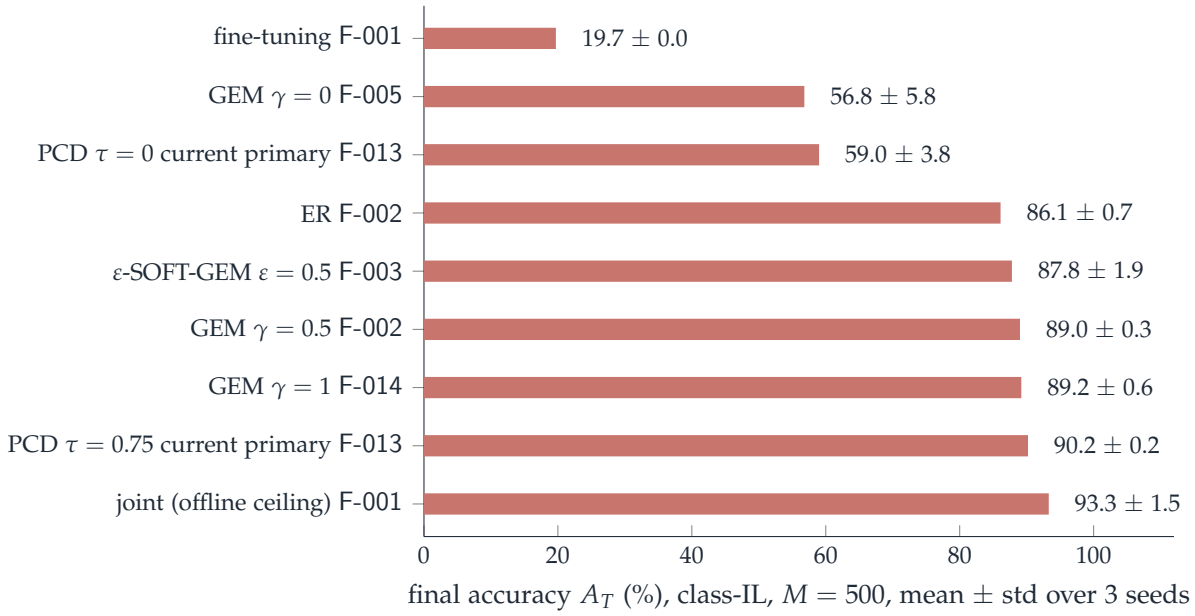


Figure 5: Split MNIST, memory $M = 500$. The PCD rows have no rescale. A-GEM (30.2%) is left out: Mammoth's A-GEM ignores the memory size, so it is not a matched baseline yet (decision D-017).

What to see:

- Fine-tuning sits at the class-IL floor (19.7%) and joint at the ceiling (93.3%).
- GEM's exact projection ($\gamma = 0$) and PCD with $\tau = 0$ and a current-only primary land in the same place (about 57–59%). They are the same algorithm.
- A large enough extra push toward old tasks recovers almost everything: GEM's $\gamma \geq 0.25$ (88.6–89.2), ε-SOFT-GEM's $\varepsilon = 0.5$ (87.8) or PCD's $\tau = 0.75$ (90.2). A small push is not enough: ε-SOFT-GEM with $\varepsilon = 0.1$ stays at 57.9 (F-003).
- The best configuration so far is PCD with $\tau = 0.75$, a current-only primary and no rescale: 90.2 ± 0.2 (F-013).

6.3 Finding 1: the margin and replay in the primary do the same job

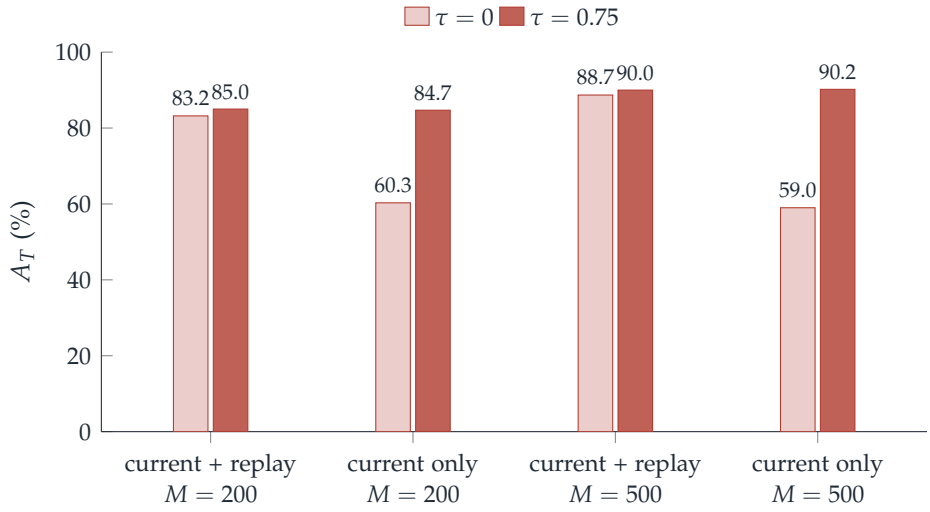


Figure 6: PCD on Split MNIST, no rescale, 3 seeds (F-012 at $M = 200$, F-013 at $M = 500$). The x-axis is what goes into the primary objective.

The most useful result so far With a current-only primary, the margin is worth +24 to +31 points. With replay already in the primary, it adds only about +1.3 to +1.8. The margin and replay-in-the-primary buy the same stability. So "τ helps" is only true relative to what is in the primary.

Mental model Replay in the primary already pulls the step toward old tasks, so the walls bind less often and need less correction (at $\tau = 0$, $M = 500$, no rescale: active on 48% of steps with a replay primary vs 72% with a current-only one, F-013). Take replay out of the primary and the walls are the only thing protecting old tasks, so the margin matters a lot. The paper story is therefore an *attribution*: the margin, replay in the primary, and GEM's γ each buy the same thing.

Two more observations from the same runs:

- The rescale step (15) does not help on Split MNIST; turning it off is slightly better (F-012).

- Normalization on or off makes almost no difference here (within 0.3 points, F-004): the losses already have similar scales.

6.4 Finding 2: PCD vs GEM's own knob

GEM's γ is the same kind of knob as τ (F-014, with $\gamma = 0$ from F-005 and $\gamma = 0.5$ from F-002): at $M = 500$, $\gamma = 0$ gives 56.8%, but $\gamma = 0.25,\ 0.5,\ 1.0$ give 88.6, 89.0, 89.2.

PCD ($\tau = 0.75$, current-only primary, no rescale) vs the best GEM γ (paired, same seeds) $+1.0 \pm 0.5$ points of A_T at $M = 500$ (per seed $+1.0$, $+0.5$, $+1.5$), and $+2.2 \pm 1.5$ at $M = 200$ (per seed $+0.8$, $+1.9$, $+3.8$) (F-013 vs GEM $\gamma = 1.0$, F-014, at $M = 500$; F-012 vs GEM $\gamma = 0.5$, F-002, at $M = 200$). Small, but the same sign on every seed. Note that $\tau = 0.75$ is above H1's stated range (0.1–0.5): inside it, PCD $\tau = 0.5$ with a replay primary beats GEM $\gamma = 0.5$ at $M = 500$ by only $+0.5 \pm 0.2$ (F-004 vs F-002).

Against ER (same PCD configuration, paired, F-002) the gap is larger: $+4.1 \pm 0.6$ ($M = 500$) and $+5.3 \pm 0.5$ ($M = 200$) points of A_T, and -5.5 and -6.8 points of forgetting, for -0.2 points of learning accuracy at both memory sizes. Against GEM the learning-accuracy cost is -0.5 to -0.7. Both are inside H1's 2-point budget.

6.5 Finding 3: on CIFAR-100, PCD is about equal to ER

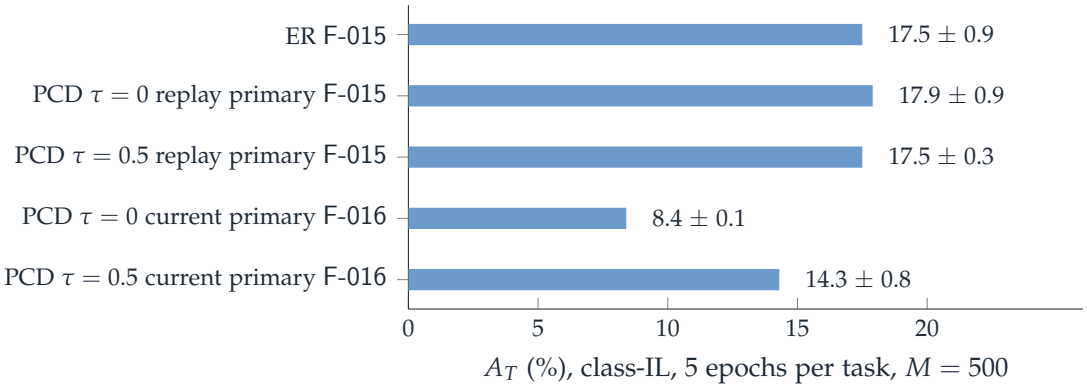


Figure 7: Split CIFAR-100, 10 tasks, 3 seeds. Class-IL CIFAR-100 is hard: every method is below 20%.

- With replay in the primary, PCD equals ER: $+0.4 \pm 0.6$ at $\tau = 0$, $+0.0 \pm 1.2$ at $\tau = 0.5$ (paired, F-015). At $\tau = 0.5$, forgetting drops by 2.7 points but learning accuracy drops by 2.6: stability bought with plasticity, net zero.
- The walls rarely bind: at $\tau = 0$ they are active on only 9.6% of steps. The replay primary already satisfies them.
- With a current-only primary, $\tau = 0$ collapses to 8.4%, close to the class-IL level the literature reports for GEM (10.7% [9], Tab. 3; to spot-check, D-008). $\tau = 0.5$ lifts it to 14.3%: about two-thirds of the gap to a replay primary. Unlike Split MNIST, the margin replaces replay only partly here (F-016).

- At 1 epoch per task (F-006, F-007) everything was undertrained: PCD 13.1–13.5%, ER 13.2%, GEM $\gamma = 0.5$ 10.3%, and joint (9.4%) was even below ER, so it was not a ceiling at that budget. That is why we moved to 5 epochs.

6.6 Scouting for Salah's objectives (1 seed, not for reporting)

RO3, the regularization form (F-009, F-011).

- In class-IL, EWC and PCD-EWC both sit at the fine-tuning floor (about 19.7%). So H2 must be read in **task-IL**, where EWC with $\lambda = 10^3$ scores 98.0% and PCD-EWC 96.0–98.6% depending on the penalty scale c.
- The ε floor breaks PCD-EWC's scale invariance at small scales: the share of the denominator taken by ε (`eps_ratio`) is 0.8–1.0 at $c \leq 1$ but 0.004 at $c = 100$. This is Rem. B.11 seen in practice.
- EWC with $\lambda = 10^4$ diverged at lr 0.1 (NaN loss). It is kept as a data point.

RO4, EMA boundary modes (F-010). A_T: `bank` 84.9, `reset` 84.8, `carry` 84.2. But the normalization error is *lower* for `carry` (0.59) than for `reset` (0.63) or `bank` (0.64). H3's proposed mechanism is not supported so far, on one seed.

6.7 Problems found and handled

- Mammoth's A-GEM ignores the memory size (it stores one batch per task), so $M = 200$ and $M = 500$ are identical. It is excluded until a matched version exists (D-017).
- Mammoth's joint baseline crashed on CIFAR-100; worked around and rerun (D-019).
- The analysis double-counted seeds when an ablation re-ran an existing configuration; it now de-duplicates per seed.

> **Reading so far** On Split MNIST, per-task walls plus a margin give a small, consistent edge over GEM and a large one over ER. On CIFAR-100 class-IL, replay in the primary already does the work and the walls rarely bind. Both are honest outcomes for a controlled study.

CHAPTER 7

7 · What Is Next

Nothing is running right now. The next batch is chosen after the meeting.

7.1 Open questions for Dr. AlHajri

1. Is a controlled study of PCD's components in CL an acceptable contribution, or should one variant be argued as a method?
2. The regularization form maps onto the paper's compression setting. Should it be the headline, with rehearsal as the secondary study?
3. FedPCD uses instantaneous ℓ_2 normalization and EMA only in an appendix. Was that EMA reset per round or kept per client? Would you expect ℓ_2 to win in CL too?
4. What are the final deadline and page limit? Is the FedPCD code available?

7.2 Next runs

- **Faris (RO2).**
  - A matched A-GEM plug-in, so A-GEM becomes a valid baseline (shared code: needs Salah's OK).
  - A finer τ grid on CIFAR-100 with a current-only primary ($\tau \in \{0.25, 0.75, 1.0\}$), to see where the partial recovery peaks.
  - A learning-rate grid for the top 3 configurations of each method family. This is the promotion gate: until it is done, no comparison is fair.
- **Salah (RO3, RO4).** Rerun the scouting on his machine as `S-*` experiments, read H2 in task-IL, and run the boundary modes with both fallbacks.
- **Both.** The agreement gate (identical ER and EWC runs on both machines must agree within ± 0.5 points). Then freeze PROTOCOL v1 and the pre-registration, rerun the promoted configurations with 3 seeds $\times$ 3 class orders, and write the paper in IEEEtran.

7.3 Timeline

The final deadline is assumed to be about 1 December; it is one of the questions for the meeting.

Week	Dates	Goal
W1	Nov 4–10	workspace, toy (RO1), plug-ins, exploration *(we are here)*
W2	Nov 11–17	agreement gate, pre-registration, H1–H3 sweeps
W3	Nov 18–24	CIFAR-100 runs of the best configurations, 3 seeds $\times$ 3 orders
W4	Nov 25–Dec 1	paper (IEEEtran), defense Q&A

Quick reference

Symbol	Meaning
g_0, $\tilde{g}_0$	primary gradient (the current task), raw and normalized
g_j, $\tilde{g}_j$	secondary gradient for old task j, raw and normalized
$\tilde{d}$, d	PCD direction before and after the rescale
τ	the margin: fraction of each old task's own progress that it must get, $\tau \in [0,1]$
λ_j	how much of $\tilde{g}_j$ is added to the primary; > 0 only when wall j is active
$c = \cos\varphi$	cosine between primary and an old task; $c < 0$ means they conflict
β, ε	EMA memory (0.999) and stability constant (10^{-8})
M	replay memory size (number of stored examples)
$R_{t,i}$	accuracy on task i after training task t
A_T, F_T, LA	final accuracy, forgetting, learning accuracy
class-IL / task-IL	pick among all classes / among the given task's classes

Result	Formula
First-order change	$L(\theta - \eta d) - L(\theta) \approx -\eta\, g^\top d$
Forgetting	$\Delta L_A \approx -\eta\, g_A^\top g_B$
PCD problem	$\min_{\tilde{d}} \frac{1}{2}\|\tilde{d} - \tilde{g}_0\|^2$ s.t. $\tilde{g}_j^\top \tilde{d} \geq \tau \|\tilde{g}_j\|^2$
One wall	$\lambda = \max\big(0, (\tau\|\tilde{g}_1\|^2 - \tilde{g}_1^\top \tilde{g}_0)/\|\tilde{g}_1\|^2\big)$; unit vectors: $\lambda = \max(0, \tau - c)$
Normalization	$v \leftarrow \beta v + (1-\beta)\|g\|^2$, $\hat{v} = v/(1-\beta^s)$, $\tilde{g} = g/\sqrt{\hat{v} + \varepsilon}$
Rescale	$d = \tilde{d}\,\|g_0\|/\|\tilde{d}\|$ ($d = 0$ if $\tilde{d} = 0$)
Threshold	for $c < 0$: primary still descends iff $\tau < (1 - c^2)/(-c)$; reachable with $\tau \leq 1$ only for $c < -0.62$
Many walls	$\tilde{d} = \tilde{g}_0 + \sum_j \lambda_j \tilde{g}_j$, $\lambda \geq 0$; dual on $M_{ij} = \tilde{g}_i^\top \tilde{g}_j$

Where to look in the repository. `docs/RESEARCH_PLAN.md` (the plan) · `docs/WALKTHROUGH.md` (guided tour) · `docs/reports/` (what ran) · `docs/DECISIONS.md` (why things are the way they are) · `src/pcd_cl/README.md` (paper-to-code map) · the board's *How it works* tab (the same math, with interactive figures).

References

[1] D. Varam and M. I. AlHajri. Not all objectives are born equal: Priority-constrained descent for hierarchical multi-objective optimization. *Transactions on Machine Learning Research*, 2026.

[2] Y. Irshaid, D. Varam and M. I. AlHajri. FedPCD: Priority-constrained federated aggregation. arXiv:2610.13508, 2026.

[3] D. Lopez-Paz and M. Ranzato. Gradient episodic memory for continual learning. *NIPS*, 2017.

[4] A. Chaudhry, M. Ranzato, M. Rohrbach and M. Elhoseiny. Efficient lifelong learning with A-GEM. *ICLR*, 2019.

[5] G. Hu, W. Zhang, H. Ding and W. Zhu. Gradient episodic memory with a soft constraint for continual learning. arXiv:2011.07801, 2020.

[6] J. Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *PNAS* 114(13), 2017.

[7] A. Chaudhry et al. On tiny episodic memories in continual learning. arXiv:1902.10486, 2019.

[8] P. Buzzega, M. Boschini, A. Porrello, D. Abati and S. Calderara. Dark experience for general continual learning: a strong, simple baseline. *NeurIPS*, 2020.

[9] D.-W. Zhou et al. Class-incremental learning: A survey. *IEEE TPAMI* 46(12), 2024.